MANUAL DE ENGENHARIA E DICIONÁRIO TÉCNICO DO CÓDIGO-FONTE

Análise Arquitetural, Mapeamento Arquivo a Arquivo e Diretrizes de Sustentação de Banca

Candidato: Andrio Gabriel Epp | **Curso:** Engenharia da Computação (Unisinos) | **Data de Defesa:** Nov/Dez 2024

---

1 Visão Geral da Arquitetura e Ciclo de Vida dos Dados

O presente projeto de graduação implementa um sistema híbrido de inteligência artificial voltado ao diagnóstico incipiente e classificação de falhas internas em transformadores de potência imersos em óleo mineral, fundamentando-se na Análise de Gases Dissolvidos (DGA – *Dissolved Gas Analysis*).

A arquitetura do projeto foi estruturada em duas trilhas experimentais rigorosamente desacopladas, denominadas **Track A** e **Track B**, projetadas para responder a duas perguntas científicas centrais:

1. **Track A (Série Temporal / Acoplamento Híbrido – Base Katser et al., 2022):** Avalia a hipótese de acoplamento temporal-estático. Uma rede neural profunda recorrente (**Stacked LSTM**) processa a evolução físico-química dos gases, gerando projeções futuras ($h = 14$ e $h = 28$ passos de 12 horas, totalizando 7 e 14 dias à frente). O vetor estático do instante de inspeção é fundido às projeções futuras e otimizado conjuntamente por um Algoritmo Genético Melhorado (**IGA**) acoplado a um classificador **XGBoost**.
2. **Track B (Classificação Estática Pura – Base Compilada IEC 60599):** Benchmark multiclasse em larga escala (7 classes de operação e defeito), comparando a otimização estocástica do **IGA-XGBoost** contra métodos tradicionais normativos (Razões IEC 60599 e Triângulo 1 de Duval) e modelos de referência do estado da arte internacional (Wu et al., 2021).

1.1 Diagrama de Fluxo e Ciclo de Vida dos Dados

O ciclo de vida dos dados opera em 6 estágios sequenciais, garantindo rastreabilidade determinística, reprodutibilidade estocástica (sementes fixas) e imunidade contra vazamento de dados (*data leakage*):

Ciclo de Vida Ponta a Ponta: Dos Dados Brutos ao Diagnóstico Explicável

1. Ingestão e Saneamento Crítico (Preprocessing):

- *Track A:* 3.000 séries temporais (420 passos × 4 gases: H_2, CO, C_2H_4, C_2H_2). Validação de hashes MD5, ausência de sobreposição e particionamento estratificado (2.100 treino/validação, 900 teste cego congelado).
- *Track B:* 2.321 amostras brutas em PPM. Aplicação do filtro de integridade estrita: expurgo de 303 duplicatas exatas e 56 amostras conflitantes (mesma assinatura de gases com classes diagnósticas divergentes). Dataset saneado com $N = 1.962$ amostras.

2. Modelagem Temporal e Projeção Preditiva (LSTM – Track A):

- Aplicação de janela deslizante ($w^* = 50$ timesteps = 25 dias). Normalização min-max independente por gás fitada estritamente no treino (`ScalerPerGas`).
- Stacked LSTM (2 camadas × 64 unidades, dropout 0.20). Previsão recursiva multi-passo ($h = 14$ e $h = 28$) gerando 8 variáveis preditivas.

3. Engenharia de Atributos Físico-Químicos (Feature Engineering):

- Extração de snapshots dos gases no tempo presente, cálculo de razões estequiométricas normativas (IEC 60599, Rogers, Doernenburg) e somas de hidrocarbonetos totais (TH, TH_1).
- Montagem do espaço vetorial: **17 atributos candidatos no Track A** e **19 atributos candidatos no Track B**.

4. Otimização Conjunta Genética (IGA – Improved Genetic Algorithm):

- Representação cromossômica híbrida (máscara binária de atributos + 9 hiperparâmetros contínuos do XGBoost normalizados em $[0, 1]$).
- Função de fitness F_1-Macro ponderada por penalidade de parcimônia. Seleção por torneio ($k = 3$), crossover uniforme ($P_c = 0.90$), mutação híbrida ($P_m = 0.10$) e elitismo de 2 indivíduos.

5. Treinamento Supervisionado e Balanceamento Termodinâmico (XGBoost + CustomSMOTE):

- No Track B: `CustomSMOTE` sintetiza vizinhos estritamente no espaço dos 5 gases brutos e recalcula determinística e não-linearmente as 14 razões e relações estequiométricas, preservando a coerência termodinâmica.
- No Track A: Tratamento do severo desbalanceamento (81% Normal) via `sample_weight` inversamente proporcional à frequência das classes.

6. Avaliação Estatística Rigorosa e Interpretabilidade Teórica (Stats & TreeSHAP):

- Avaliação com 5 sementes pseudoaleatórias independentes e bootstrap não-paramétrico ($B = 5.000$ iterações) com intervalos de confiança BCa (95%).
- Decomposição exata de Shapley em tempo polinomial ($\mathcal{O}(TLD^2)$) na escala multi-logloss, mapeando a importância global, curvas de dependência e migração de crédito informacional.

1.2 Stack Tecnológico e Justificativas de Engenharia

Tecnologia / Pacote	Versão de Referência	Justificativa Técnica e de Engenharia
Python	3.10+	Ecossistema padrão de computação científica; tipagem estática e suporte assíncrono.
TensorFlow / Keras	2.15+	Implementação da Stacked LSTM com otimizador Adam vetorial e suporte a tensores 3D tridimensionais (N, W, F).
XGBoost	2.0.3+	Algoritmo de Gradient Boosted Decision Trees de alta performance com suporte nativo a `tree_method='hist'` e regularização L1/L2.
Imbalanced-Learn	0.12+	Base para a implementação da classe customizada `CustomSMOTE` acoplada ao `ImbPipeline`.
Scikit-Learn	1.4+	Primitivas de validação cruzada estratificada repetida (`RepeatedStratifiedKFold`), cálculo de métricas e escalonadores.
SHAP	0.45.0+	Implementação do `TreeExplainer` fundamentada na teoria dos jogos cooperativos (axiomas de Shapley: Eficiência, Simetria, Dummy e Aditividade).
SciPy / NumPy	1.12+ / 1.26+	Estruturas de dados vetoriais em C/Fortran, testes não-paramétricos de Wilcoxon e rotinas de bootstrap.

2 Dicionário Técnico de Arquivos e Módulos

A seguir, apresenta-se o inventário técnico completo e aprofundado dos 34 arquivos que compõem o repositório `tcc-transformer-fault`.

2.1 Configuração Global e Constantes de Domínio

`src/config.py` – Núcleo Central de Configurações e Hiperparâmetros

Papel no TCC: Estabelece o ponto único de verdade (*single source of truth*) para diretórios, sementes estocásticas, limites físicos dos gases, classes normativas e hiperparâmetros de busca, impedindo divergências de parâmetros entre módulos independentes.
Entradas e Saídas:

- *Inputs:* Constantes literais e variáveis de ambiente; caminhos do sistema operacional resolvidos via `pathlib.Path`.
- *Outputs:* Exportação de constantes globais: `RANDOM_SEED = 42`, tuplas de gases (`KATSER_GASES`, `CLASSIFICATION_GASES`), mapeamentos de rótulos (`KATSER_CLASSES`, `MAPA_ROTULOS_ZH`), sementes de avaliação (`SEEDS_AVALIACAO = [42, 123, 456, 789, 2024]`), parâmetros da LSTM e do IGA. Cria automaticamente as pastas `data/processed`, `reports/metrics` e `reports/figures`.

Funcionamento Técnico e Mecânica Interna: Define explicitamente a separação entre os 7 gases canônicos da IEC 60599 citados conceitualmente no TCC I e as limitações das bases empíricas reais: a base Katser possui 4 gases (H_2, CO, C_2H_4, C_2H_2), enquanto a base chinesa possui 5 gases ($H_2, CH_4, C_2H_6, C_2H_4, C_2H_2$). Define as razões normativas calculáveis e as somas de hidrocarbonetos totais ($TH = CH_4 + C_2H_4 + C_2H_2$ e $TH_1 = TH + C_2H_6$).
Integração com o Restante do Sistema: É importado por praticamente todos os módulos do pipeline. Nenhuma constante mágica (*magic number*) é instanciada fora deste arquivo.
Pontos Críticos para a Banca:

- *Questionamento:* "Por que o TCC I menciona 7 gases e o código opera com 4 e 5 gases?"
- *Defesa Técnica:* A norma IEC 60599 define 7 gases combustíveis de referência. Contudo, transformadores reais em subestações operam com sensores com número restrito de colunas cromatográficas. Nenhuma base de dados aberta mundial possui os 7 gases canônicos simultaneamente. O código foi desacoplado de forma pragmática para refletir fielmente as bases físicas disponíveis sem comprometer a validade teórica.

2.2 Subprocesso de Pré-processamento e Validação de Dados

`src/preprocessing/validate_katser_data.py` – Validação Estrutural da Base Katser

Papel no TCC: Executa uma auditoria de qualidade pré-voo (*pre-flight integrity check*) na base de dados de séries temporais de Katser et al. (2022) antes do treinamento da rede LSTM.
Entradas e Saídas:

- *Inputs:* 3.000 arquivos CSV em `data/external/katser_npp_transformers/` e metadados de rótulos (`labels_fdd_train.csv`, `labels_fdd_test.csv`, `labels_rul_*.csv`).
- *Outputs:* Relatório JSON detalhado em `reports/metrics/katser_validation_report.json`.

Funcionamento Técnico e Mecânica Interna: Executa 6 testes estruturais: (1) checagem de vazamento por interseção de IDs entre conjuntos; (2) consistência biunívoca entre IDs de falha (FDD) e vida útil remanescente (RUL); (3) correspondência física de arquivos em disco vs. registros das tabelas; (4) busca exata de duplicatas por cálculo de hash criptográfico MD5 sobre o binário dos CSVs; (5) amostragem estocástica ($n = 300$) para detecção de quase-duplicatas via coeficiente de correlação de Pearson de H_2 ($r > 0,999$); (6) análise de separabilidade das classes via deltas médios ($\Delta = \mu_{t\in[410,420]} - \mu_{t\in[0,10]}$).
Integração com o Restante do Sistema: Pré-requisito metodológico executado antes de `preprocess_katser.py`.
Pontos Críticos para a Banca: Atesta a integridade da partição oficial de teste fornecida pela Skoltech/Rosatom. Garante que os 900 transformadores do teste jamais foram expostos ao pipeline de calibração.

`src/preprocessing/preprocess_katser.py` – Ingestão e Split Estratificado da Base Katser

Papel no TCC: Realiza o parsing dos 3.000 CSVs da base Katser, aplica o particionamento estratificado do conjunto de desenvolvimento e persiste as estruturas em tensores NumPy serializados.
Entradas e Saídas:

- *Inputs:* 2.100 séries de treino e 900 séries de teste oficial (cada série com $T = 420$ passos e 4 gases).
- *Outputs:* Arquivos `katser_train.npz` ($1.785 \times 420 \times 4$), `katser_val.npz` ($315 \times 420 \times 4$) e `katser_test.npz` ($900 \times 420 \times 4$), além de seus respectivos vetores de rótulos de classe $y \in \{1, 2, 3, 4\}$.

Funcionamento Técnico e Mecânica Interna: Aplica divisão aleatória estratificada de 15% nas 2.100 séries de desenvolvimento, indexada pela categoria de falha (`category`). Preserva estritamente intacto o conjunto de 900 séries de teste cego. O particionamento é efetuado por transformador (série inteira), assegurando que timesteps de uma mesma máquina não sejam distribuídos entre treino e validação.
Integração com o Restante do Sistema: Depende de `config.py`; alimenta diretamente o `data_loader.py` da LSTM e a extração de features em `build_katser_features.py`.
Pontos Críticos para a Banca:

- *Questionamento:* "Por que o split entre séries foi aleatório estratificado e não cronológico?"
- *Defesa Técnica:* Cada CSV representa um transformador físico distinto em operação na usina. Os identificadores numéricos não possuem ordenação cronológica entre diferentes equipamentos. A cronologia temporal (*walk-forward*) é mandatória e aplicada rigorosamente **dentro** de cada série temporal de 420 passos pelo módulo `windowing.py`.

`src/preprocessing/preprocess_classification.py` – Saneamento e Split do Track B

Papel no TCC: Processa a base de dados chinesa multiclasse compilada em Excel, executa a política de desduplicação e saneamento e particiona os dados para o experimento estático do IGA-XGBoost.
Entradas e Saídas:

- *Inputs:* Arquivo `dataset_2321_7classes.xlsx` (2.321 amostras brutas com rótulos em ideogramas chineses).
- *Outputs:* `china_train.csv` (1.372×6), `china_val.csv` (295×6) e `china_test.csv` (295×6), armazenando os 5 gases em ppm bruto e a coluna `fault_type`.

Funcionamento Técnico e Mecânica Interna: 1. Tradução unívoca dos rótulos chineses para o padrão IEC 60599 (7 classes: Normal, PD, D1, D2, T1, T2, T3). 2. Algoritmo de desduplicação estrita:

$$\text{Assinatura } \mathbf{x} = (\mathrm{H_2}, \mathrm{CH_4}, \mathrm{C_2H_6}, \mathrm{C_2H_4}, \mathrm{C_2H_2})$$

- Duplicatas exatas com mesmo rótulo: preserva a primeira ocorrência e remove as excedentes (303 amostras eliminadas). - Duplicatas conflitantes (mesmo vetor $\mathbf{x}$, rótulos y divergentes): remoção completa de todas as ocorrências conflitantes (56 amostras eliminadas), pois violam a propriedade funcional de aprendizado supervisionado ($f(\mathbf{x}) \to y$). 3. Particionamento estratificado 70/15/15 mantendo proporções das classes minoritárias (PD e Normal).
Integração com o Restante do Sistema: Alimenta `build_classification_features.py`.
Pontos Críticos para a Banca: A literatura prévia (e.g. Wu et al., 2021) treinava sobre essa base com duplicatas, gerando vazamento cruzado entre treino e teste e inflacionando artificialmente as métricas acima de 99%. O saneamento deste trabalho reduziu o volume para 1.962 amostras, restabelecendo a honestidade científica do benchmark.

`src/preprocessing/windowing.py` – Gerador de Janelas Deslizantes Temporais

Papel no TCC: Transforma séries temporais contínuas unidimensionais por transformador em tensores tridimensionais supervisionados de entrada e saída para a rede neural recorrente.
Entradas e Saídas:

- *Inputs:* Série temporal contínua $\mathbf{S} \in \mathbb{R}^{T \times F}$ ou batch $\mathbf{B} \in \mathbb{R}^{N \times T \times F}$, tamanho de janela w, horizonte h.
- *Outputs:* Tensores de entrada $\mathbf{X} \in \mathbb{R}^{M \times w \times F}$ e saída $\mathbf{y} \in \mathbb{R}^{M \times F}$ (quando $h = 1$).

Funcionamento Técnico e Mecânica Interna: Implementa o protocolo estrito de janelamento *walk-forward* sem sobreposição entre transformadores. Para uma série de comprimento T, gera $M = T - w - h + 1$ janelas. Para $T = 420, w = 50, h = 1$, obtém-se $M = 370$ amostras por transformador. Vetoriza o fatiamento via `np.stack`,

evitando laços interpretados lentos.
Integração com o Restante do Sistema: Utilizado em `data_loader.py` para o treino da LSTM e no teste de inferência em `multi_horizon_eval.py`.
Pontos Críticos para a Banca: O corte temporal garante ausência de vazamento do futuro para o passado (*look-ahead bias*). O alvo $\mathbf{y}$ é sempre estritamente posterior à janela temporal $[t-w+1, t]$.

`src/preprocessing/run_preprocessing.py` – Orquestrador do Pré-processamento

Papel no TCC: Ponto de entrada CLI (*Command-Line Interface*) para execução sequencial e automatizada de todo o pipeline de pré-processamento de ambas as trilhas.
Entradas e Saídas: Acionado via linha de comando (`python -m src.preprocessing.run_preprocessing`); coordena a execução de `validate_katser_data.py`, `preprocess_katser.py` e `preprocess_classification.py`. Gera logs estruturados em console e valida existência de saídas.
Funcionamento Técnico e Mecânica Interna: Garante a ordem correta de dependências e bloqueia o pipeline caso ocorra falha de validação ou assert em qualquer dataset.
Integração com o Restante do Sistema: Prepara todo o diretório `data/processed` para as Fases 3 (LSTM) e 4 (Feature Engineering).

2.3 Subprocesso da Rede Neural Temporal (LSTM)

`src/lstm/data_loader.py` – Carregamento e Normalização Segregada da LSTM

Papel no TCC: Encapsula a lógica de carga dos splits Katser, conversão em janelas deslizantes e, criticamente, implementa a normalização min-max segregada por gás.
Entradas e Saídas:

- *Inputs:* Arquivos `katser_{train, val, test}.npz`.
- *Outputs:* Gerador/Tensores normalizados $(\mathbf{X}_{tr}, \mathbf{y}_{tr})$, $(\mathbf{X}_{val}, \mathbf{y}_{val})$, $(\mathbf{X}_{te}, \mathbf{y}_{te})$ e o objeto `ScalerPerGas`.

Funcionamento Técnico e Mecânica Interna: Contém a classe customizada `ScalerPerGas`. O escalonamento linear padrão global falha na DGA porque as ordens de grandeza entre os gases diferem drasticamente (CO atinge milhares de ppm enquanto C_2H_2 ocorre em dezenas de ppm). O `ScalerPerGas` computa:

$$x^{(g)}_{\text{norm}} = \frac{x^{(g)} - \min_{\text{treino}}(x^{(g)})}{\max_{\text{treino}}(x^{(g)}) - \min_{\text{treino}}(x^{(g)}) + \epsilon}, \quad \forall g \in \{\text{H}_2, \text{CO}, \text{C}_2\text{H}_4, \text{C}_2\text{H}_2\}$$

Os parâmetros $(\min, \max)$ são calculados **exclusivamente sobre o conjunto de treino** (1.785×420 pontos) e aplicados para transformar validação e teste.
Integração com o Restante do Sistema: Usado por `trainer.py`, `evaluator.py`, `multi_horizon_eval.py` e exportado para o `build_katser_features.py`.
Pontos Críticos para a Banca: Impede contaminação de dados. Se o escalonador visse os máximos da partição de teste, haveria *data leakage* de escala.

`src/lstm/model.py` – Arquitetura Deep Stacked LSTM

Papel no TCC: Define e compila a topologia da rede neural recorrente bidimensional empilhada com regularização por Dropout para regressão temporal multivariada.
Entradas e Saídas:

- *Inputs:* Parâmetros estruturais w (tamanho de janela) e $n_{features} = 4$.
- *Outputs:* Instância compilada de modelo Keras (`tf.keras.Model`).

Funcionamento Técnico e Mecânica Interna: 1. Fixação determinística global de sementes estocásticas (`fixar_seeds`) sincronizando Python, NumPy, hash de ambiente e TensorFlow Graph Engine. 2. Arquitetura Keras Sequential:

$$\mathbf{X} \in \mathbb{R}^{w \times 4} \longrightarrow \text{LSTM}_1(64, \text{return_sequences=True}) \longrightarrow \text{Dropout}(0.2) \longrightarrow \text{LSTM}_2(64) \longrightarrow \text{Dense}(4, \text{linear})$$

A primeira camada captura dinâmicas temporais de curto e médio prazo, propagando a sequência completa de estados ocultos $\mathbf{h}_t \in \mathbb{R}^{64}$; a segunda consolida o vetor de contexto temporal final $\mathbf{h}_w \in \mathbb{R}^{64}$, projetado na camada linear de 4 neurônios correspondente aos gases no instante $t+1$.

3. Otimizador Adam ($\alpha = 0,001$) com função de perda Erro Quadrático Médio (MSE).
Integração com o Restante do Sistema: Invocado por `trainer.py` e carregado por `build_katser_features.py`.
Pontos Críticos para a Banca: A ativação da camada de saída deve ser estritamente linear, pois concentrações normalizadas são contínuas em $[0, 1]$. O uso de 2 camadas empilhadas com 64 unidades demonstrou capacidade ótima de representação sem incorrer em sobreajuste na validação.

`src/lstm/trainer.py` – Motor de Treinamento da LSTM

Papel no TCC: Conduz o loop de aprendizado supervisionado da rede neural com paradas antecipadas e preservação dos pesos ótimos.
Entradas e Saídas:

- *Inputs:* Modelo compilado, tensores de treino e validação, batch size (1.024), épocas máximas (200).
- *Outputs:* Modelo treinado em memória, histórico de treinamento (`History.history`) e métricas de convergência.

Funcionamento Técnico e Mecânica Interna: Utiliza callback `EarlyStopping` monitorando a perda na partição de validação (`val_loss`) com paciência de 15 épocas e tolerância mínima $\delta = 10^{-6}$. Configura `restore_best_weights=True`, revertendo os tensores de pesos da rede para o ponto de mínimo global de validação caso ocorra degradação por sobreajuste.
Integração com o Restante do Sistema: Chamado por `run_lstm.py`.
Pontos Críticos para a Banca: O batch size relativamente elevado (1.024) foi adotado para acelerar o processamento das mais de 660.000 janelas deslizantes sem perda de estabilidade do gradiente estocástico.

`src/lstm/evaluator.py` – Avaliação Quantitativa Multivariada da LSTM

Papel no TCC: Avalia a precisão preditiva do modelo nos conjuntos de teste e validação, calculando métricas físicas nos dados desnormalizados.
Entradas e Saídas:

- *Inputs:* Modelo treinado, tensores normalizados, objeto `ScalerPerGas`.
- *Outputs:* Dicionário de métricas globais e por gás: $\mathrm{MAE}, \mathrm{RMSE}, \mathrm{MAPE}$ e R^2.

Funcionamento Técnico e Mecânica Interna: Inverte a transformação min-max nos alvos e previsões via `scaler.inverse_transform`, retornando os gases à escala física de PPM antes de calcular os erros. Impede distorções decorrentes da escala normalizada.
Integração com o Restante do Sistema: Salva relatórios em `reports/metrics/lstm_grid_search_results.json`.
Pontos Críticos para a Banca: As métricas são reportadas em PPM real. Demonstrar que a LSTM alcançou $R^2 \geq 0,99$ no horizonte imediato ($h = 1$).

`src/lstm/multi_horizon_eval.py` – Avaliação de Degradação Recursiva Multi-Horizonte

Papel no TCC: Avalia formalmente o limite de previsibilidade temporal da LSTM em até 16 horizontes operacionais futuros ($h \in [1, 168]$ timesteps, equivalente a 12 horas até 12 semanas).
Entradas e Saídas:

- *Inputs:* Modelo treinado na Fase 3, dados de teste Katser.
- *Outputs:* Métricas por horizonte em JSON e gráficos de degradação: `lstm_multi_horizon_r2_norm.png` e `lstm_multi_horizon_mae_pct_norm.png`.

Funcionamento Técnico e Mecânica Interna: Implementa previsão recursiva multi-passo:

$$\hat{\mathbf{y}}_{t+1} = f(\mathbf{x}_{t-w+1:t}); \quad \hat{\mathbf{y}}_{t+2} = f([\mathbf{x}_{t-w+2:t}, \hat{\mathbf{y}}_{t+1}]); \quad \dots$$

O erro recursivo se acumula conforme o horizonte avança. O script computa o ponto em que o R^2 médio cruza o patamar de confiabilidade de $0,80$, determinando os limites práticos de projeção para a engenharia de atributos ($h = 14$ e $h = 28$).
Integração com o Restante do Sistema: Justifica experimentalmente a escolha dos horizontes $h = 14$ (7 dias) e $h = 28$ (14 dias) utilizados em `build_katser_features.py`.
Pontos Críticos para a Banca: Mostra que a LSTM mantém $R^2 > 0,85$ até $h = 70$ (5 semanas), comprovando que as projeções para 7 e 14 dias são tecnicamente sólidas e não meros ruídos acumulados.

src/lstm/run_lstm.py – Grid Search e Treinamento Final da LSTM

Papel no TCC: Executa o grid search sobre o tamanho da janela de observação temporal ($w \in \{10, 20, 30, 50\}$), seleciona o melhor hiperparâmetro w^* e realiza o treinamento final com múltiplas sementes.
Entradas e Saídas:

- *Inputs:* Dados processados da base Katser.
- *Outputs:* Artefatos do modelo persistidos: `lstm_best_model_norm.keras`, `lstm_scaler_norm.pkl`, `lstm_best_config_norm.json`.

Funcionamento Técnico e Mecânica Interna: Conduz a validação cruzada sobre as janelas candidatas, confirmando a superioridade de $w^* = 50$ (menor MAE e maior R^2). Treina o modelo final na configuração ótima e consolida os artefatos serializados.
Integração com o Restante do Sistema: Entrega os modelos congelados para a extração de features na Fase 4.

2.4 Subprocesso de Engenharia de Atributos e Acoplamento Híbrido

src/feature_engineering/build_katser_features.py – Construção de Atributos do Track A

Papel no TCC: Constrói a matriz final de atributos tabulares para o modelo XGBoost no Track A, realizando a fusão do vetor estático atual com as projeções temporais geradas pela LSTM.
Entradas e Saídas:

- *Inputs:* Splits Katser (`katser_*.npz`), modelo LSTM congelado e seu escalonador.
- *Outputs:* Arquivos `katser_features_train.csv` (1.785×18), `katser_features_val.csv` (315×18), `katser_features_test.csv` (900×18) e o escalonador `katser_feature_scaler.pkl`.

Funcionamento Técnico e Mecânica Interna: Gera um espaço vetorial de 17 atributos candidatos divididos em 3 grupos funcionais:

1. **Grupo 1 – Snapshot Observado (4 gases):** Últimos valores reais medidos no instante $t = 420$ (H_2, CO, C_2H_4, C_2H_2).
2. **Grupo 2 – Razões Físico-Químicas e Somas (5 atributos):**
   - C_2H_2/C_2H_4: Razão canônica da IEC 60599 (discrimina falhas elétricas por arco vs. térmicas).
   - H_2/C_2H_2: Razão adaptada (alta sensibilidade a descargas parciais na ausência de CH_4).
   - $TH_{katser} = C_2H_4 + C_2H_2$: Hidrocarbonetos totais parciais disponíveis.
   - C_2H_2/TH_{katser} e C_2H_4/TH_{katser}: Proporções relativas normalizadas.
3. **Grupo 3 – Projeções Recursivas da LSTM (8 atributos):** 4 gases previstos para $h = 14$ (7 dias) e 4 gases para $h = 28$ (14 dias).

Aplica divisão segura por zero (`razao_segura`) retornando 0,0 caso o denominador seja nulo, impedindo surgimento de `inf` ou `NaN`. Ajusta um `MinMaxScaler` estritamente nos dados de treino.
Integração com o Restante do Sistema: Alimenta os scripts de treinamento `train_hybrid.py` e `fase55_track_a.py`.
Pontos Críticos para a Banca: Detalha por que a razão CO/C_2H_4 foi excluída: não possui respaldo normativo nem termodinâmico na literatura DGA, sendo produzidos por pirólises de materiais distintos (celulose vs. óleo).

src/feature_engineering/build_classification_features.py – Construção de Atributos do Track B

Papel no TCC: Constrói a matriz tabular para o benchmark multiclasse na base chinesa, calculando o conjunto completo de razões e relações estequiométricas normativas.
Entradas e Saídas:

- *Inputs:* Arquivos `china_{train, val, test}.csv`.
- *Outputs:* Arquivos `china_features_train.csv`, `china_features_val.csv`, `china_features_test.csv` e o pipeline de normalização `china_feature_scaler.pkl`.

Funcionamento Técnico e Mecânica Interna: Constrói 19 atributos candidatos:

1. **Gases Brutos em PPM (5 atributos):** $H_2, CH_4, C_2H_6, C_2H_4, C_2H_2$.

2. **Razões Normativas (5 atributos):** CH_4/H_2, C_2H_2/C_2H_4, C_2H_4/C_2H_6 (as 3 razões da IEC 60599), C_2H_2/CH_4 e C_2H_2/C_2H_6 (Doernenburg / Wu et al., 2021).
3. **Somas e Percentuais de Wu et al. (9 atributos):** $\text{TH} = CH_4 + C_2H_4 + C_2H_2$, $\text{TH}_1 = \text{TH} + C_2H_6$, frações de CH_4, C_2H_4, C_2H_2 sobre TH, e frações de $CH_4, C_2H_6, C_2H_4, C_2H_2$ sobre TH_1.

Pipeline de normalização fita um pré-processamento não-linear composto por transformação logarítmica estável seguida de MinMax:

$$\mathbf{z} = \ln(1 + \mathbf{x}) \implies \mathbf{z}_{\text{norm}} = \text{MinMaxScaler}(\mathbf{z})$$

Esse pipeline é fitado estritamente no treino e serializado em `pickle`.
Integração com o Restante do Sistema: Fornece a matriz para o IGA em `train_classification.py`.
Pontos Críticos para a Banca: A transformação $\log(1 + x)$ comprime a cauda pesada característica das distribuições de gases em DGA (onde concentrações variam de $0,1$ a mais de 10.000 ppm), evitando saturação de gradientes e instabilidade nos cortes de decisão.

`src/feature_engineering/run_feature_engineering.py` – Orquestrador da Fase 4

Papel no TCC: Script de orquestração via linha de comando para gerar as matrizes de atributos do Track A e Track B em uma única chamada controlada.
Entradas e Saídas: Disparador CLI que invoca `build_katser_features.py` e `build_classification_features.py`.
Funcionamento Técnico e Mecânica Interna: Executa os geradores de atributos, checa se as dimensões esperadas foram geradas em disco e registra o relatório de conclusão.
Integração com o Restante do Sistema: Garante a prontidão de todos os arquivos `.csv` em `data/processed` para a execução dos algoritmos genéticos.

2.5 Subprocesso de Otimização Genética e Classificação (IGA-XGBoost)

`src/iga_xgboost/cromossomo.py` – Representação e Decodificação do Cromossomo Híbrido

Papel no TCC: Define a estrutura genômica do indivíduo no Algoritmo Genético Melhorado, integrando a seleção de subconjuntos de atributos e o ajuste fino de hiperparâmetros do classificador em uma única entidade matemática otimizável.
Entradas e Saídas:

- *Inputs:* Vetor contínuo unidimensional $\mathbf{c} \in [0, 1]^L$, onde $L = N_{\text{features}} + 9$.
- *Outputs:* Máscara booleana de atributos ativos e dicionário de hiperparâmetros mapeados para a escala real do XGBoost.

Funcionamento Técnico e Mecânica Interna: Estrutura do cromossomo:

- $\mathbf{c}[0 : N_{\text{features}}]$: Parte binária (limiarizada em $0,5$ ou probabilística na inicialização). $1 \implies$ atributo incluído; $0 \implies$ atributo descartado.
- $\mathbf{c}[N_{\text{features}} : L]$: Parte contínua com 9 genes normalizados em $[0, 1]$.

Espaço de busca dos hiperparâmetros (`HPARAM_SPECS`):

1. `learning_rate` $\in [0.01, 0.30]$ (float linear)
2. `max_depth` $\in [3, 12]$ (inteiro)
3. `n_estimators` $\in [50, 500]$ (inteiro)
4. `subsample` $\in [0.5, 1.0]$ (float linear)
5. `colsample_bytree` $\in [0.3, 1.0]$ (float linear)
6. `gamma` $\in [0.0, 5.0]$ (float linear – ganho mínimo de perda para split)
7. `min_child_weight` $\in [1, 10]$ (inteiro)
8. `reg_alpha` $\in [0.0, 1.0]$ (float linear – regularização L1)
9. `reg_lambda` $\in [0.0, 5.0]$ (float linear – regularização L2)

Inicialização Enviesada por Domínio: Ao inicializar a população, as razões canônicas da IEC 60599 recebem probabilidade de ativação $P = 0,80$, enquanto atributos brutos recebem $P = 0,50$, acelerando a busca com

heurística prévia de engenharia elétrica.
Integração com o Restante do Sistema: Utilizado pelo motor `iga.py` e por todos os scripts de treino de classificação.
Pontos Críticos para a Banca: O tamanho do cromossomo varia por trilha: $L = 17 + 9 = 26$ genes no Track A; $L = 19 + 9 = 28$ genes no Track B; e $L = 18 + 9 = 27$ genes no baseline de Wu.

`src/iga_xgboost/iga.py` – Motor do Algoritmo Genético Melhorado (IGA)

Papel no TCC: Implementa o algoritmo bioinspirado que realiza a busca estocástica global no espaço hiperdimensional de atributos e hiperparâmetros, maximizando o desempenho preditivo.
Entradas e Saídas:

- *Inputs:* Dados de treino e validação, parâmetros evolutivos (população $P = 50$, gerações máximas $G = 100$, parada antecipada de 20 gerações sem melhoria).
- *Outputs:* Melhor cromossomo global $\mathbf{c}^*$, histórico de convergência geracional (fitness máximo e médio) e tempo de execução.

Funcionamento Técnico e Mecânica Interna: 1. **Função de Fitness com Penalidade de Parcimônia:**

$$\text{Fitness}(\mathbf{c}) = F_{1,\text{macro}}(\text{val}) - \lambda_p \cdot \frac{\sum_{i=1}^{N_f} c_i}{N_f}$$

onde $\lambda_p = 0,001$. Se nenhuma feature for ativada na máscara binária, atribui penalidade extrema Fitness $= -1,0$.
2. **Seleção por Torneio:** $k = 3$ indivíduos sorteados aleatoriamente; o indivíduo de maior fitness é selecionado para reprodução.
3. **Crossover Uniforme Híbrido ($P_c = 0,90$):** Para cada gene, troca aleatoriamente com probabilidade $0,5$ entre os pais.
4. **Mutação Diferenciada ($P_m = 0,10$):** - Nos genes binários (features): bit-flip ($0 \leftrightarrow 1$). - Nos genes contínuos (hiperparâmetros): perturbação gaussiana $\mathcal{N}(0; \sigma^2)$ com $\sigma = 0,10$, seguida de truncamento estrito no intervalo $[0, 1]$.
5. **Elitismo Estrito:** Os 2 indivíduos de maior fitness de cada geração são clonados diretamente para a próxima população sem sofrer crossover ou mutação.
6. **Cache de Fitness Memoizado:** Utiliza tabela hash para armazenar o fitness de genótipos já avaliados, eliminando o custo computacional de treinar modelos XGBoost idênticos.
Integração com o Restante do Sistema: Invocado por `train_hybrid.py`, `train_classification.py` e `train_wu2021_baseline.py`.
Pontos Críticos para a Banca: A parada antecipada após 20 gerações sem ganho poupa horas de processamento computacional. A convergência geralmente ocorre entre a 25ª e a 45ª geração.

`src/iga_xgboost/train_classification.py` – Treinamento e Otimização do Track B

Papel no TCC: Conduz a otimização evolutiva e a validação do IGA-XGBoost no Track B (Experimento C3) e implementa a classe fundamental `CustomSMOTE`.
Entradas e Saídas:

- *Inputs:* `china_features_{train, val, test}.csv` e o escalonador serializado.
- *Outputs:* `iga_track_b_results.json`, histórico de evolução `iga_evolution_track_b.json` e o modelo serializado `iga_best_model_track_b.pkl`.

Funcionamento Técnico e Mecânica Interna: A Classe CustomSMOTE (Preservação Termodinâmica): O SMOTE tradicional falha na DGA se interpolar diretamente razões e percentuais (uma interpolação linear entre duas razões produz valores termodinamicamente impossíveis que violam a conservação estequiométrica). O `CustomSMOTE` resolve isso em 4 passos:

1. Isola estritamente as primeiras 5 colunas correspondentes aos gases brutos em ppm ($X_{\text{gases}} = X[:, :5]$).
2. Aplica o algoritmo SMOTE ($k = 5$) no subespaço de dimensão 5, gerando amostras sintéticas plausíveis de gases dissolvidos.
3. Desfaz a normalização via inversa do scaler e $\exp(x) - 1$, obtendo concentrações físicas em PPM real.
4. Recalcula matematicamente e de forma determinística todas as 14 razões, somas e percentuais sobre os gases sintéticos, aplicando a normalização final.

Executa o IGA, treina o modelo final com 5 sementes no teste e avalia a matriz de confusão e F_1-Macro.
Integração com o Restante do Sistema: Gera o modelo C3 para as análises de explicabilidade SHAP na Fase

6.
Pontos Críticos para a Banca: O IGA selecionou 16 de 19 atributos, descartando apenas C_2H_2, C_2H_2/CH_4 e $\%CH_4/TH_1$, atingindo $F_1 = 0,7974$ no teste cego.

`src/iga_xgboost/train_hybrid.py` – Treinamento e Otimização do Track A (Sistema Híbrido)

Papel no TCC: Executa o acoplamento temporal-estático final no Track A, otimizando conjuntamente as features da LSTM e hiperparâmetros do XGBoost.
Entradas e Saídas:

- *Inputs:* `katser_features_{train, val, test}.csv`.
- *Outputs:* Relatório `iga_track_a_results.json`, `iga_evolution_track_a.json` e `iga_best_model_track_a.pkl`.

Funcionamento Técnico e Mecânica Interna: Trata o severo desbalanceamento de classes da base Katser (81% Normal) através de ponderação de perda `sample_weight` inversamente proporcional à frequência das classes:

$$w_i = \frac{N}{K \cdot N_{y_i}}$$

Roda o IGA para selecionar os melhores atributos dentre os 17 candidatos e tunar o XGBoost. Executa a avaliação em 5 sementes independentes na partição de 900 transformadores do teste.
Integração com o Restante do Sistema: Fornece a base empírica para o experimento C1 e para a ablação de ausência da LSTM em `fase55_iga_sem_lstm.py`.
Pontos Críticos para a Banca: O IGA selecionou 8 atributos ótimos (incluindo 2 projeções da LSTM: H_2 em $h = 14$ e C_2H_4 em $h = 28$). Atingiu $F_{1,\text{macro}} = 0,7604 \pm 0,0012$.

`src/iga_xgboost/train_wu2021_baseline.py` – Reprodução do Modelo de Referência de Wu et al. (2021)

Papel no TCC: Reproduz com máxima fidelidade metodológica o modelo de referência internacional de Wu et al. (2021) sobre o mesmo dataset saneado, viabilizando comparação científica direta e imparcial.
Entradas e Saídas:

- *Inputs:* Dados da base chinesa saneada.
- *Outputs:* Métricas em `reports/metrics/baseline_wu2021_results.json`.

Funcionamento Técnico e Mecânica Interna: Constrói as exatas 18 features tabulares especificadas na Tabela 1 do artigo de Wu. Roda o IGA sobre os 18 atributos com os parâmetros de busca padronizados do projeto ($P = 50, G = 100$), treina o XGBoost resultante e avalia nas 5 sementes no teste.
Integração com o Restante do Sistema: Utilizado na Seção de Resultados do TCC e nos testes de significância estatística de `fase55_track_b.py`.
Pontos Críticos para a Banca: O modelo de Wu alcançou $F_1 = 0,7659 \pm 0,0099$ no dataset saneado, comprovando que o IGA-XGBoost do TCC ($F_1 = 0,7974$) o superou com significância estatística ($p = 0,0002$ no teste pareado de Wilcoxon).

`src/iga_xgboost/traditional_methods.py` – Implementação Normativa das Razões IEC e Triângulo de Duval

Papel no TCC: Implementa as duas principais metodologias diagnósticas clássicas da indústria elétrica (IEC 60599 e Triângulo 1 de Duval), servindo como linha de base (*baseline*) de domínio.
Entradas e Saídas:

- *Inputs:* Concentrações de gases em PPM bruto.
- *Outputs:* Diagnóstico normativo em 6 classes de falha ou rótulos de não-classificação (`ND` para fora de faixas e `DT` para zonas mistas do triângulo).

Funcionamento Técnico e Mecânica Interna: 1. **Método das Razões IEC 60599:2015:** Calcula $R_1 = C_2H_2/C_2H_4$, $R_2 = CH_4/H_2$, $R_3 = C_2H_4/C_2H_6$. Aplica a tabela clássica de faixas lógicas ($< 0,1$, $0,1 - 1$, $1 - 3$, > 3). Casos fora da tabela recebem rótulo `ND`.
2. **Triângulo 1 de Duval:** Converte as concentrações relativas de CH_4, C_2H_4, C_2H_2 em percentuais sobre a soma e aplica teste geométrico de inclusão de ponto em polígonos convexos nos eixos baricêntricos (`matplotlib.path.Path.contains_point`) para mapear as 6 zonas normativas e a zona mista `DT`.
Avaliação Justa: Reporta resultados em dois cenários: "somente falhas" (6 classes) e "7 classes". Casos `ND` e `DT`

contam rigorosamente como erro diagnóstico.
Integração com o Restante do Sistema: Executado nas comparações estatísticas da Fase 5 e Fase 5.6.
Pontos Críticos para a Banca: Métodos normativos não foram criados para triagem de normalidade (possuem recall 0% para a classe Normal, pois a norma exige presença comprovada de excesso de gás antes da aplicação das razões).

`src/iga_xgboost/run_iga_xgboost.py` – Orquestrador do Pipeline de Classificação

Papel no TCC: Ponto de entrada CLI para rodar a Fase 5 completa (Track A híbrido, Track B multiclasse, baselines normativos e baseline de Wu).
Entradas e Saídas: Dispara `train_hybrid.py`, `train_classification.py` e `train_wu2021_baseline.py`.
Funcionamento Técnico e Mecânica Interna: Controla o fluxo de execução das baterias genéticas e assegura que todos os arquivos de métricas JSON sejam gerados.
Integração com o Restante do Sistema: Finaliza a etapa de modelagem preditiva, liberando os dados para a avaliação da Fase 5.5 e Fase 6.

`src/iga_xgboost/generate_figures.py`, `generate_bar_chart.py`, `generate_rich_plots.py` – Geração Gráfica

Papel no TCC: Suíte de renderização gráfica automatizada responsável pela geração de todas as figuras de alta resolução (300 DPI) inseridas na monografia do TCC.
Entradas e Saídas:

- *Inputs:* JSONs de métricas e históricos evolutivos em `reports/metrics/`.
- *Outputs:* Arquivos `.png` em `reports/figures/` (curvas de convergência do fitness, matrizes de confusão normalizadas, gráficos de barras com barras de erro de desvio-padrão e radar plots multivariados).

Funcionamento Técnico e Mecânica Interna: Utiliza `matplotlib` e `seaborn` configurados com fontes vetoriais, grid limpo e paleta de cores consistente com as normas da ABNT.

2.6 Subprocesso de Avaliação Estatística, Validação Cruzada e SHAP

`src/evaluation/stats_utils.py` – Primitivas de Estatística Não-Paramétrica e Bootstrap

Papel no TCC: Núcleo de utilitários estatísticos avançados para testes de hipóteses pareados e intervalos de confiança não-paramétricos.
Entradas e Saídas:

- *Inputs:* Vetores de predições pareadas no teste e vetores de métricas ao longo de sementes.
- *Outputs:* Intervalos de confiança de 95% computados via Bootstrap BCa (ou percentil), p-valores de Wilcoxon e testes de McNemar.

Funcionamento Técnico e Mecânica Interna: 1. `bootstrap_modelos`: Executa reamostragem com reposição ($B = 5.000$ iterações) no conjunto de teste, recomputando a métrica em cada réplica para construir a distribuição empírica da diferença ($\Delta F_1 = F_{1,A} - F_{1,B}$). Determina se o intervalo contém o zero.
2. Teste pareado dos postos sinalizados de Wilcoxon (`scipy.stats.wilcoxon`) para comparar modelos sob múltiplas sementes.
3. Teste de McNemar para comparar tabelas de contingência binária de concordância/discordância entre dois classificadores no mesmo conjunto de teste.
Integração com o Restante do Sistema: Invocado pelos scripts `fase55_*.py` e `fase56.py`.
Pontos Críticos para a Banca: Elimina a subjetividade de comparar apenas médias pontuais; fornece respaldo inferencial rigoroso para a defesa perante a banca.

`src/evaluation/fase55_track_a.py` – Avaliação Estatística e Ablação do Track A

Papel no TCC: Executa o benchmarking definitivo do Track A, confrontando estatisticamente o modelo Híbrido C1 (com LSTM) contra o modelo Default C2 e a Ablação C5a (sem LSTM).
Entradas e Saídas:

- *Inputs:* CSVs de atributos do Katser e modelo persistido.
- *Outputs:* Relatório JSON consolidado `reports/metrics/fase55_track_a.json`.

Funcionamento Técnico e Mecânica Interna: Calcula métricas agregadas sob 5 sementes pseudoaleatórias independentes e roda o teste de Wilcoxon. Comprova que:

$$F_{1,\text{macro}}(\text{C1}) = 0,7604 \pm 0,0012 \quad \text{vs.} \quad F_{1,\text{macro}}(\text{C5a}) = 0,7602 \pm 0,0015 \quad (p = 0,4375)$$

A diferença não é estatisticamente significativa no teste cego.
Integração com o Restante do Sistema: Sustenta a conclusão empírica sobre o desacoplamento temporal no texto do TCC.
Pontos Críticos para a Banca: O valor $p = 0,4375$ (maior que $\alpha = 0,05$) refuta com precisão estatística a hipótese de ganho prático da inclusão de previsões de gases em transformadores estáveis da base Katser.

`src/evaluation/fase55_track_b.py` – Avaliação Estatística e Benchmarking do Track B

Papel no TCC: Executa o confronto estatístico multiclasse no Track B, comparando o IGA-XGBoost contra o XGBoost Default, contra o modelo de Wu et al. e contra os métodos normativos IEC e Duval.
Entradas e Saídas:

- *Inputs:* Dados do Track B e modelos treinados.
- *Outputs:* Relatório JSON consolidado `reports/metrics/fase55_track_b.json`.

Funcionamento Técnico e Mecânica Interna: Roda bootstrap de 5.000 amostras e testes pareados de Wilcoxon. Demonstra a superioridade do IGA-XGBoost ($F_1 = 0,7974 \pm 0,0073$) sobre: - XGBoost Default ($F_1 = 0,7748 \pm 0,0081$, $p = 0,0002$); - Baseline Wu et al. ($F_1 = 0,7659 \pm 0,0099$, $p = 0,0002$); - IEC 60599 Razões ($F_1 = 0,3967$, $p < 0,0001$); - Triângulo 1 de Duval ($F_1 = 0,5513$, $p < 0,0001$).
Integração com o Restante do Sistema: Fornece a tabela principal de resultados quantitativos do Capítulo 4 do TCC.

`src/evaluation/fase55_iga_sem_lstm.py` – Experimento de Ablação Isolada da LSTM

Papel no TCC: Conduz a ablação estrita da LSTM no Track A: executa o IGA restrito unicamente aos 9 atributos estáticos e razões físicas, sem qualquer acesso aos 8 atributos previstos pela rede neural.
Entradas e Saídas:

- *Inputs:* `katser_features_{train, val, test}.csv`.
- *Outputs:* Relatório `reports/metrics/iga_track_a_sem_lstm_results.json`.

Funcionamento Técnico e Mecânica Interna: Otimiza conjuntamente a seleção dentre as 9 features estáticas e os 9 hiperparâmetros do XGBoost. Mostra que o modelo atinge $F_1 = 0,7602$ utilizando apenas o snapshot estático e razões físicas.
Integração com o Restante do Sistema: Complementa `fase55_track_a.py` para fechar a prova científica de ablação.

`src/evaluation/fase56.py` – Verificações de Robustez e Sanidade Pré-Feature Freeze

Papel no TCC: Consolida 4 verificações metodológicas de profundidade para blindar o trabalho contra questionamentos técnicos severos de banca examinadora.
Entradas e Saídas:

- *Inputs:* Dados e rotinas de avaliação de ambas as trilhas.
- *Outputs:* Relatório JSON detalhado em `reports/metrics/fase56.json`.

Funcionamento Técnico e Mecânica Interna: 1. **Baseline Bruto (Track B):** Treina XGBoost exclusivamente sobre os 5 gases brutos em ppm ($F_1 = 0,7821$). Prova que a engenharia de atributos eleva o teto informacional para $0,7974$.
2. **Matriz Cruzada Rótulo vs. Norma:** Cruza amostras reais de defeito com as coordenadas geométricas do Triângulo de Duval, comprovando que quase metade das falhas térmicas T3 (alta temperatura) do dataset chinês apresentam composições químicas típicas de T1/T2, explicando os erros sistemáticos das regras normativas.
3. **Testes Unitários Duval e IEC:** Avalia a implementação das regras clássicas em pontos canônicos publicados em artigos de validação, comprovando matematicamente acurácia de 100% na lógica de fronteiras.
4. **Validação Cruzada Estratificada Repetida (5x5 Repeated K-Fold):** Executa 25 splits independentes (5 folds × 5 repetições) com hiperparâmetros congelados, atestando estabilidade estrutural ($F_1 = 0,7479 \pm 0,0198$) e descartando que os resultados sejam anomalia de uma partição específica.
Integração com o Restante do Sistema: Constitui o principal escudo metodológico contra contestações de

viés amostral.
Pontos Críticos para a Banca: Mostra domínio estatístico avançado e honestidade na caracterização de imperfeições dos dados de campo.

`src/evaluation/fase6_shap_track_a.py` – Interpretabilidade TreeSHAP do Track A

Papel no TCC: Aplica a teoria dos jogos cooperativos via algoritmo TreeSHAP para explicar as decisões do modelo híbrido e desvendar o mecanismo interno de migração de crédito informacional.
Entradas e Saídas:

- *Inputs:* Modelo híbrido C1 congelado e partição de teste (900×8).
- *Outputs:* Figuras de importância global (`fig1_shap_summary_bar_track_a.png`), beeswarm (`fig2_shap_beeswarm_track_a.png`) e matriz de interação (`fig3_shap_interaction_matrix_track_a.png`).

Funcionamento Técnico e Mecânica Interna: Utiliza `shap.TreeExplainer` configurado para árvores de regressão logística aditiva multiclasse (*multi-logloss scale*). Calcula os valores exatos de Shapley ϕ_i para cada uma das 900 amostras e 4 classes. Revela que o snapshot observado de H_2 detém 43% da importância global, enquanto as projeções futuras da LSTM (lstm_h14_H2 e lstm_h28_C2H4) recebem contribuição marginal ($< 3\%$).
Integração com o Restante do Sistema: Fornece a justificativa mecanicista para o desfecho da hipótese temporal.
Pontos Críticos para a Banca: O TreeSHAP demonstra empiricamente a **migração de crédito**: como a correlação entre os gases no instante presente e futuro é extrema ($r > 0,997$ devido à inércia físico-química do óleo mineral), as árvores de decisão atribuem prioridade ao corte no atributo presente, esvaziando o ganho marginal das variáveis preditivas.

`src/evaluation/fase6_shap_track_b.py` – Interpretabilidade Multiclasse TreeSHAP do Track B

Papel no TCC: Mapeia os mecanismos internos de decisão do IGA-XGBoost multiclasse no Track B, identificando os discriminadores físicos primários de cada tipo de falha elétrica e térmica.
Entradas e Saídas:

- *Inputs:* Modelo C3 congelado e conjunto de teste (295×16).
- *Outputs:* Beeswarm plots por classe em `reports/figures/fase6/` e métricas de contribuição média absoluta $|\phi|$.

Funcionamento Técnico e Mecânica Interna: Decompõe a contribuição individual de cada uma das 16 features selecionadas para as 7 classes de saída. Comprova concordância estrita com a termodinâmica de quebra de óleo da IEC 60599:

- **Descargas de Alta Energia (D2):** C_2H_2/C_2H_4 e C_2H_2/TH são as variáveis dominantes com empuxo positivo na função logit.
- **Falhas Térmicas de Média/Alta Temperatura (T2 e T3):** Dominadas pela razão C_2H_4/C_2H_6 e fração de etileno C_2H_4/TH.
- **Descargas Parciais (PD):** Determinadas preponderantemente pelo gás hidrogênio (H_2).

Integração com o Restante do Sistema: Conecta a teoria da física de dielétricos aos pesos internos das árvores de decisão.
Pontos Críticos para a Banca: Prova definitiva de que o modelo não é uma "caixa preta" ininteligível, mas sim um extrator alinhado aos princípios termodinâmicos conhecidos da engenharia elétrica.

`src/evaluation/fase6_shap_extras.py` – Gráficos Locais de Força, Waterfall e Limiares Físicos

Papel no TCC: Gera explicações locais de alta resolução para amostras individuais (casos canônicos de acerto e falsos positivos críticos) e extrai limiares físicos de transição via regressão não-linear LOWESS.
Entradas e Saídas:

- *Inputs:* Explainer TreeSHAP e instâncias de teste específicas.
- *Outputs:* Gráficos de força (*Force Plots*) e gráficos em cascata (*Waterfall Plots*) em `reports/figures/fase6/fig5_force_*.png`, além de curvas de dependência com ajuste LOWESS.

Funcionamento Técnico e Mecânica Interna: 1. Isola predições canônicas com probabilidade máxima ($P > 0,95$) para cada classe e decompõe aditivamente o somatório das contribuições locais:

$$\ln\left(\frac{P(Y=c)}{1-P(Y=c)}\right) = \phi_0(c) + \sum_{i=1}^{M} \phi_i(c)$$

2. Analisa falsos positivos entre sobreaquecimentos térmicos (e.g., T1 predito como T2), demonstrando visualmente quais atributos empurraram a decisão na direção incorreta.
3. Ajusta curvas de regressão localmente ponderada (LOWESS – *Locally Weighted Scatterplot Smoothing*) na diagonal dos valores de interação SHAP ($\Phi_{i,i}$), descobrindo empiricamente os pontos de inflexão onde uma razão gasosa passa a sinalizar defeito.
Integração com o Restante do Sistema: Enriquece a análise qualitativa do Capítulo 4 da monografia.
Pontos Críticos para a Banca: Demonstra capacidade de auditoria preditiva individual: o engenheiro operador de subestação pode justificar tecnicamente por que o modelo recomendou a abertura emergencial de um transformador.

2.7 Suíte de Testes Unitários e Achado Crítico de Auditoria

`tests/test_lstm.py` – Auditoria de Código e Testes da LSTM

Papel no TCC: Arquivo originalmente planejado para conter a suíte de testes unitários automatizados da rede neural recorrente (verificação de shapes tensoariais, consistência do `windowing.py` e paridade de saídas).
Achado Crítico de Auditoria (Ponto de Máxima Atenção):

Alerta de Auditoria: Arquivo Corrompido por Download Intersticial

Durante a auditoria exaustiva do repositório, identificou-se que o arquivo `tests/test_lstm.py` foi baixado inadvertidamente como uma página HTML de aviso de verificação de vírus do Google Drive (*"Google Drive - Virus scan warning"*) em vez do código Python executável original.

Mecânica do Ocorrido: O download automatizado via script/curl atingiu a página intersticial gerada pelo Google Drive para arquivos de script potencialmente executáveis (9.5 kB), salvando o cabeçalho HTML em disco.
Impacto Metodológico no TCC: Nenhum impacto nos resultados ou nas métricas científicas da pesquisa. Todos os testes e validações de integridade da LSTM foram executados e validados por módulos dedicados em `src/preprocessing/validate_katser_data.py`, `src/lstm/evaluator.py` e pelas rotinas de teste unitário canônico contidas em `src/evaluation/fase56.py`.
Postura de Defesa perante a Banca: Se arguido sobre a ausência de testes unitários em `tests/`, o candidato deve declarar com transparência total:

> *"Os testes estruturais da LSTM foram incorporados de forma integrada nos módulos funcionais `evaluator.py` e `validate_katser_data.py`. O arquivo em `tests/` sofreu corrupção na etapa de sincronização de repositório via Google Drive, mas a validação computacional foi integralmente sustentada pelo protocolo da Fase 5.6."*

3 Tópicos de Defesa Metodológica e Perguntas de Banca

A seguir, estruturam-se os 5 tópicos metodológicos mais sensíveis do trabalho, acompanhados de suas fundamentações científicas completas e do gabarito ideal de argumentação técnica para a defesa perante a banca examinadora.

3.1 Tópico 1: O Fenômeno de Deduplicação e Saneamento da Base Estática (Track B)

Fundamentação Científica: O Vazamento Oculto na Literatura de DGA

Contexto do Problema: O dataset compilado de DGA com 2.321 registros (`dataset_2321_7classes.xlsx`) é amplamente utilizado em artigos da literatura chinesa e internacional (incluindo Wu et al., 2021), nos quais são reportadas acurácias rotineiramente superiores a 98% e até 99,5%.
A Investigação Metodológica do TCC: Ao submeter a base a uma auditoria estrita de unicidade vetorial no espaço $\mathbb{R}^5$, nosso trabalho descobriu que:

1. **303 amostras eram cópias exatas (duplicatas idênticas)** de outros registros com a mesma classe diagnóstica.
2. **56 amostras eram duplicatas estritamente conflitantes**: vetores de concentrações de gases $\mathbf{x}$ rigorosamente idênticos até a quarta casa decimal, mas associados a diagnósticos totalmente divergentes (ex.: o mesmo vetor classificado como Normal em uma subestação e como Descarga de Baixa Energia D1 em outra).

Total de registros espúrios: **359 amostras (15,5% do volume total bruto)**.
O Impacto Científico do Saneamento: Quando um dataset contendo 15,5% de duplicatas é submetido a um particionamento aleatório usual de treino e teste (70/30 ou 80/20), centenas de amostras de teste tornam-se clones exatos de amostras presentes no conjunto de treino. O classificador não aprende a generalizar fronteiras termodinâmicas; ele simplesmente memoriza instâncias (*data leakage*). Ao purgar as 359 amostras espúrias, o dataset foi fixado em $N = 1.962$ amostras únicas. O desempenho do modelo de Wu et al. despencou de alegados $> 98\%$ para $76,59\%$, e o modelo IGA-XGBoost proposto atingiu $79,74\%$.

Sabatina de Banca: Pergunta 1

Examinador: *"Ao remover 359 amostras da base chinesa, você não reduziu excessivamente o conjunto de treinamento, prejudicando o poder estatístico do seu classificador? Por que não manteve as duplicatas para aumentar o suporte das classes minoritárias?"*
Gabarito de Defesa Ideal:
"Pelo contrário, membro da banca. Manter duplicatas em aprendizado de máquina supervisionado viola a premissa fundamental de que as partições de treino e teste devem ser variáveis aleatórias independentes e identicamente distribuídas (i.i.d.). A presença de 303 clones idênticos gerava vazamento cruzado severo (*data leakage*), fazendo com que o conjunto de teste avaliasse memorização em vez de generalização. Mais crítico ainda: a presença de 56 amostras conflitantes introduzia contradição determinística nos dados, pois a mesma assinatura físico-química de gases recebia múltiplos rótulos contraditórios, o que matematicamente impede qualquer estimador consistente de convergir para a regra de Bayes ótima. A literatura que relata acurácias de 99% sobre essa base sofre de viés de otimismo artificial decorrente desse vazamento. Ao purgar 15,5% da base, o trabalho abriu mão de métricas infladas em prol da integridade científica, demonstrando que a acurácia real em dados limpos é da ordem de 79% a 80%, representando o verdadeiro teto informacional dos 5 gases da norma IEC 60599."

3.2 Tópico 2: O Desfecho da Hipótese de Acoplamento Temporal no Track A

Fundamentação Científica: Inércia Físico-Química e o Ganho Marginal da LSTM

A Hipótese Inicial da Pesquisa: Prever a trajetória dos gases dissolvidos a médio prazo ($h = 14$ e $h = 28$ passos $\times$ 12h = 7 e 14 dias) através de uma Stacked LSTM forneceria ao XGBoost uma visão dinâmica antecipada da taxa de evolução temporal (dC/dt), elevando expressivamente o F_1-Macro em relação ao uso de um snapshot estático isolado.
A Constatação Empírica no Teste Cego (900 transformadores):

- Modelo C1 (Híbrido: Snapshot + Razões + Previsões LSTM): $F_{1,\text{macro}} = \mathbf{0,7604 \pm 0,0012}$
- Modelo C5a (Ablação: Snapshot + Razões, SEM LSTM): $F_{1,\text{macro}} = \mathbf{0,7602 \pm 0,0015}$
- Ganho líquido marginal: $\Delta F_1 = +0,0002$ $(+0,02\%)$.

- Teste pareado dos postos sinalizados de Wilcoxon: $p = 0,4375$ ($p > 0,05 \implies$ não significativo).

A Explicação Físico-Química e a Migração de Crédito via TreeSHAP: A rede LSTM funcionou com excelência preditiva estrita ($R^2 > 0,85$ em $h = 14$ e $h = 28$). O motivo pelo qual o XGBoost não se beneficiou das previsões decorre da física do transformador de potência imerso em óleo: 1. **Inércia de Difusão no Óleo Mineral:** O volume de óleo mineral em transformadores de grande porte varia de 10 a mais de 80 toneladas. A difusão gasosa e a solubilização são processos termodinamicamente lentos. Na base Katser (equipamentos sob operação contínua sem falhas súbitas por arco violento), o coeficiente de autocorrelação entre o gás no tempo t e no tempo $t + 14$ é $r > 0,997$. 2. **Mecanismo de Migração de Crédito no TreeSHAP:** O algoritmo TreeSHAP revelou que o atributo obs_H_2 detém 43% do ganho de informação das árvores. Como a previsão lstm_h14_H2 é quase perfeitamente colinear com o valor presente ($r > 0,997$), qualquer divisão (*split*) que a árvore pudesse fazer na variável prevista é absorvida de forma redundante pelo valor observado presente.

Sabatina de Banca: Pergunta 2

Examinador: *"Se a inclusão da LSTM gerou um ganho marginal de apenas 0,02% no F1-Macro sem significância estatística, a sua arquitetura híbrida não falhou? Não teria sido melhor remover a LSTM do TCC e defender apenas um modelo estático?"*
Gabarito de Defesa Ideal:
"Não, membro da banca. Em ciência da computação e engenharia, um resultado negativo rigorosamente comprovado é tão valioso quanto um ganho numérico positivo. Muitos artigos propõem arquiteturas híbridas complexas encadeando LSTM e Gradient Boosting de forma puramente teórica, mas nunca submetem a arquitetura a um teste de ablação pareado com testes de significância estatística para provar se o ganho é real ou mero ruído amostral. Nós provamos duas coisas fundamentais: Primeiro, que a LSTM isoladamente é viável e atinge $R^2 > 0,85$ até 5 semanas à frente, o que possui alto valor prático para o operador da subestação acompanhar a tendência de geração gasosa. Segundo, provamos empiricamente e via interpretabilidade TreeSHAP o mecanismo da **migração de crédito**: a altíssima inércia química do óleo mineral faz com que o snapshot do instante de inspeção já contenha quase toda a entropia necessária para a classificação do tipo de defeito em transformadores com evolução gradual. Portanto, a pesquisa documenta com integridade científica que acoplar LSTM a árvores de decisão em regime de falha incipiente lenta é redundante, recomendando o uso da LSTM para rastreamento de tendência e do XGBoost estático para o diagnóstico pontual."

3.3 Tópico 3: Preservação da Consistência Termodinâmica no CustomSMOTE

Fundamentação Científica: A Quebra de Leis Físicas por Interpolação Ingênua

O Problema da Abordagem Padrão de Oversampling em DGA: Em sistemas com severo desbalanceamento de classes, a aplicação trivial da biblioteca `imblearn.over_sampling.SMOTE` interpola vetores sintéticos diretamente na matriz completa de atributos:

$$\mathbf{x}_{\text{novo}} = \mathbf{x}_i + \lambda(\mathbf{x}_{zi} - \mathbf{x}_i), \quad \lambda \sim \mathcal{U}(0,1)$$

Se a matriz contém razões normativas como $R_1 = C_2H_2/C_2H_4$, a interpolação gera:

$$R_{1,\text{novo}} = R_{1,i} + \lambda(R_{1,zi} - R_{1,i})$$

Contudo, para os gases sintéticos brutos gerados pela mesma equação:

$$\frac{C_2H_{2,\text{novo}}}{C_2H_{4,\text{novo}}} = \frac{C_2H_{2,i} + \lambda(C_2H_{2,zi} - C_2H_{2,i})}{C_2H_{4,i} + \lambda(C_2H_{4,zi} - C_2H_{4,i})} \neq R_{1,\text{novo}}$$

A álgebra não-linear prova que a razão entre as interpolações lineares de numerador e denominador **jamais** coincide com a interpolação linear da razão. A abordagem ingênua introduz no conjunto de treinamento dados que violam as leis estequiométricas fundamentais da física de gases.
A Solução de Engenharia: A Classe CustomSMOTE: Nossa implementação no arquivo `train_classification.py` desacopla o espaço de geração do espaço de atributos: 1. Realiza o sobre-amostragem sintética estritamente sobre os 5 gases brutos em ppm ($X[:, :5]$). 2. Desnormaliza os pontos sintéticos para a escala física. 3. Recalcula matematicamente todas as 14 razões e relações de soma a partir dos gases gerados. 4. Aplica o escalonador final para re-normalização.

Sabatina de Banca: Pergunta 3

Examinador: *"No pipeline do Track B, por que você se preocupou em implementar uma classe CustomSMOTE em vez de utilizar o SMOTE padrão diretamente do scikit-learn ou imbalanced-learn? Qual é a implicação prática dessa escolha para o aprendizado do modelo?"*

Gabarito de Defesa Ideal:

"Excelente pergunta, membro da banca. Se aplicássemos o SMOTE padrão sobre o vetor completo de 19 features, estaríamos realizando interpolação linear diretamente sobre razões químicas como C_2H_2/C_2H_4 e percentuais de hidrocarbonetos totais como CH_4/TH. Matematicamente, a função razão $f(a, b) = a/b$ é uma hipérbole não-linear. A combinação linear de razões não equivale à razão das combinações lineares dos gases:

$$\lambda \frac{a_1}{b_1} + (1 - \lambda) \frac{a_2}{b_2} \neq \frac{\lambda a_1 + (1 - \lambda) a_2}{\lambda b_1 + (1 - \lambda) b_2}$$

Isso significa que o SMOTE padrão sintetiza exemplos de treino fisicamente inconsistentes, onde o valor registrado na coluna da razão diverge matematicamente da divisão dos gases individuais daquela mesma amostra sintética. O classificador seria treinado sobre dados que violam a conservação estequiométrica, aprendendo padrões artificiais impossíveis na engenharia real. Ao criar o `CustomSMOTE`, garantimos que a interpolação ocorra exclusivamente no espaço físico dos 5 gases em PPM, e que todas as 14 variáveis derivadas sejam recalculadas analiticamente a posteriori. Com isso, preservamos a coerência termodinâmica do conjunto de treinamento e a interpretabilidade real das árvores do XGBoost."

3.4 Tópico 4: A Arquitetura de Regularização do XGBoost no Espaço do IGA

Fundamentação Científica: Controle Estocástico de Complexidade em Árvores Profundas

O Comportamento Encontrado pelo IGA: Na busca global realizada pelo algoritmo genético no Track B (Experimento C3), a solução ótima convergida encontrou os seguintes hiperparâmetros para o XGBoost:

- Profundidade máxima de árvore: max_depth = **12** (o limite superior absoluto do espaço de busca!).
- Subamostragem de colunas por árvore: colsample_bytree = **0,34** (apenas 34% das variáveis por árvore).
- Subamostragem de linhas por árvore: subsample = **0,74** (amostragem estocástica de 74% do treino).
- Regularização L2 nos pesos das folhas: reg_lambda = **3,94** (penalidade severa).
- Regularização L1: reg_alpha = **0,08**.
- Ganho mínimo para expansão de nó: gamma = **1,44**.
- Taxa de aprendizado: learning_rate = **0,11**.

A Análise de Engenharia do Equilíbrio Viés-Variância: Em árvores de decisão convencionais, uma profundidade de 12 níveis com apenas 1.372 amostras de treino causaria sobreajuste (*overfitting*) imediato e catastrófico (a árvore teria capacidade teórica de criar até $2^{12} = 4.096$ folhas, isolando cada amostra individualmente). Contudo, o IGA encontrou um mecanismo compensatório sofisticado: 1. Ao forçar colsample_bytree = 0,34, cada árvore tem acesso a apenas 5 ou 6 atributos aleatórios das 16 disponíveis. As árvores tornam-se fracamente correlacionadas entre si (descorrelacionamento de ensemble). 2. O parâmetro $\gamma = 1,44$ exige que qualquer divisão reduza a perda da função objetivo em pelo menos 1,44 unidades log-loss para ser aceita, podando ativamente ramos profundos irrelevantes. 3. O termo de penalidade $\lambda = 3,94$ contrai o vetor de pesos das folhas $w_j = -G_j/(H_j + \lambda)$ em direção a zero, suavizando a probabilidade logística final.

Sabatina de Banca: Pergunta 4

Examinador: *"No Track B, o IGA selecionou max_depth = 12 para o XGBoost. Para um dataset tabular de menos de 2.000 amostras, uma profundidade de 12 não é um indicativo claro de sobreajuste aos dados de treinamento? Por que o modelo generalizou bem no teste cego?"*

Gabarito de Defesa Ideal:

"Essa é uma preocupação muito pertinente, professor. Em uma única árvore de decisão cartesiana ou em um Random Forest sem regularização, profundidade 12 causaria sobreajuste severo. No entanto, no algoritmo XGBoost com regularização de segunda ordem, a profundidade máxima é apenas um limite de teto superior (*cap*), não a profundidade efetiva média das árvores. O IGA descobriu uma sinergia de múltiplos regularizadores que compensam essa profundidade: Primeiro, fixou colsample_bytree = 0,34, de modo que cada divisão concorre com apenas um terço das variáveis disponíveis, gerando alta aleatoriedade e descorrelação entre as árvores. Segundo, configurou o

parâmetro $\gamma = 1,44$, que impõe uma poda severa pré-divisão: qualquer ramo cuja redução de perda não exceda 1,44 é compulsoriamente abortado pela equação de ganho da árvore. Terceiro, impôs $\lambda = 3,94$ como penalidade L2 no denominador do peso das folhas, amortecendo a magnitude das previsões. A prova empírica de que não houve sobreajuste é a convergência estável entre o F_1 de validação cruzada no 5x5 Repeated K-Fold $(0,7479 \pm 0,0198)$ e o teste cego final $(0,7974)$, além do intervalo de confiança estreito demonstrado pelo bootstrap de 5.000 iterações."

3.5 Tópico 5: Blindagem contra Vazamento de Dados e Protocolo Out-of-Fold

Fundamentação Científica: O Encadeamento Operacional Correto no Pipeline

A Estrutura de Blindagem Metodológica: Para assegurar a validade formal dos resultados, todo o pipeline supervisionado opera sob 4 preceitos invioláveis de engenharia de software científico:

1. **Congelamento do Conjunto de Teste Cego:** As partições de teste (900 séries no Track A e 295 amostras no Track B) foram geradas e isoladas no primeiro dia de trabalho e jamais foram passadas para o IGA, para o cálculo de médias/desvios dos escalonadores ou para a seleção de atributos.
2. **Encadeamento via ImbPipeline:** No Track B, o sobre-amostragem sintética (`CustomSMOTE`) e a seleção de atributos (`FeatureSelector`) foram acoplados via pipeline da biblioteca `imblearn`:

   ```
   ImbPipeline([('smote', CustomSMOTE()), ('select', FeatureSelector()), ('clf', XGBClassifier())])
   ```

   Durante os folds de validação cruzada do IGA, o SMOTE é aplicado **estritamente dentro da partição de treino de cada iteração**. A validação interna opera sempre sobre dados reais puros desbalanceados.
3. **Ajuste de Escalares Restrito:** O `ScalerPerGas` na LSTM e o `MinMaxScaler` tabular foram ajustados (`.fit()`) exclusivamente na partição de treino. A transformação (`.transform()`) é aplicada passivamente em validação e teste.
4. **Validação Cruzada Aninhada (Nested Cross-Validation):** Avaliação formal com 5 repetições e 5 dobras (`5x5 RepeatedStratifiedKFold`) na Fase 5.6, comprovando estabilidade estatística sob 25 partições distintas.

Sabatina de Banca: Pergunta 5

Examinador: *"Como você pode provar perante esta banca examinadora que o seu algoritmo genético não sofreu overfitting ao conjunto de validação durante as 100 gerações de busca dos hiperparâmetros?"*
Gabarito de Defesa Ideal:
"Podemos comprovar isso por três evidências metodológicas independentes implementadas na arquitetura: A primeira é o isolamento físico estrito: o conjunto de teste cego foi mantido congelado em arquivo separado e nenhuma rotina de cálculo de fitness do IGA teve acesso a ele durante as 100 gerações. A função de fitness avaliava unicamente o F_1-Macro na partição interna de validação. A segunda é a penalidade de parcimônia explícita na função objetivo do IGA:

$$\text{Fitness} = F_{1,\text{macro}}(\text{val}) - 0,001 \cdot \frac{\text{Features Ativas}}{N_{\text{total}}}$$

Essa penalidade combateu a inclusão oportunista de atributos espúrios que gerassem pequenos ganhos artificiais na validação. A terceira evidência é o experimento da Fase 5.6, onde submetemos a melhor configuração encontrada a um protocolo de Validação Cruzada Estratificada Repetida 5x5 (25 dobras com sementes alternadas). O desvio-padrão obtido foi inferior a $0,02$ ($F_1 = 0,7479 \pm 0,0198$), e o desempenho no teste cego alcançou $0,7974$, descartando qualquer sobreajuste de busca e comprovando a capacidade genuína de generalização do modelo proposto."

FIM DO MANUAL TÉCNICO E ARQUITETURAL – DOCUMENTO OFICIAL PARA DEFESA DE TCC